Day 32 : Event Loop in JS

JavaScript's Single-Threaded Nature: First Principles

Let me build this up from scratch with code examples.

1. What Does "Single-Threaded" Mean?

First Principle: Your JavaScript code runs on ONE call stack. Only ONE thing executes at a time.

```
console.log('First');
console.log('Second');
console.log('Third');

// Output:
// First
// Second
// Third
```

This ALWAYS runs in order. JavaScript can't execute `Second` and `Third` simultaneously. It's like a single-lane road - cars must go one at a time.

2. Synchronous = Blocking

First Principle: Each line blocks the next line until it completes.

```
function slowFunction() {
  // Simulate slow work
  let sum = 0;
  for (let i = 0; i < 3000000000; i++) {
    sum += i;
```

```
  }
  return sum;
}

console.log('Start');
slowFunction(); // This BLOCKS everything
console.log('End'); // This waits for slowFunction to finish
```

Your browser freezes during `slowFunction()`. You can't click buttons, scroll, or do anything. This is the problem.

3. Web APIs: JavaScript's Helpers

First Principle: The browser provides APIs that run OUTSIDE JavaScript's single thread.

```
console.log('1. Start');

// setTimeout is a Web API, NOT JavaScript
setTimeout(() => {
  console.log('2. Inside timeout');
}, 2000);

console.log('3. End');

// Output:
// 1. Start
// 3. End
// (2 seconds later)
// 2. Inside timeout
```

What happened?

- JavaScript calls `setTimeout`
- The BROWSER takes over the timer (not JavaScript)
- JavaScript continues to line 3 immediately

- After 2 seconds, browser says "hey, run this callback"

4. The Event Loop: The Coordinator

First Principle: The event loop connects the single-threaded JavaScript with async Web APIs.

```
console.log('1. Synchronous');

setTimeout(() => {
  console.log('2. Timeout 0ms');
}, 0);

Promise.resolve().then(() => {
  console.log('3. Promise');
});

console.log('4. Synchronous');

// Output:
// 1. Synchronous
// 4. Synchronous
// 3. Promise
// 2. Timeout 0ms
```

Why this order? Let me show you the mechanism:

```
// CALL STACK (JavaScript's single thread)
// [main code runs here, one line at a time]

// WEB APIs (Browser's territory)
// [setTimeout timers, fetch requests, DOM events]

// TASK QUEUES (Waiting area)
```

```
// - Microtask Queue (Promises) - HIGHER PRIORITY
// - Macrotask Queue (setTimeout, setInterval) - LOWER PRIORITY
```

5. Step-by-Step Event Loop Example

```
console.log('A');

setTimeout(() => console.log('B'), 0);

Promise.resolve()
  .then(() => console.log('C'))
  .then(() => console.log('D'));

console.log('E');

// Output: A, E, C, D, B
```

The Process:

1. **Call Stack:** Execute `console.log('A')` → Output: "A"
2. **Call Stack:** See `setTimeout` → Send to Web API → Continue
3. **Call Stack:** See `Promise.resolve()` → Add to Microtask Queue → Continue
4. **Call Stack:** Execute `console.log('E')` → Output: "E"
5. **Call Stack Empty!** Event loop checks:
   - Microtask Queue first: Execute `C` → Output: "C"
   - Another microtask: Execute `D` → Output: "D"
   - Macrotask Queue: Execute `B` → Output: "B"

6. Common Web APIs

```
// Timer APIs
setTimeout(() => {}, 1000);
```

```
setInterval(() => {}, 1000);

// Network APIs
fetch('https://api.example.com')
  .then(response => response.json());

// DOM Events
document.getElementById('btn').addEventListener('click', () => {
  console.log('Clicked!');
});

// File APIs
fs.readFile('file.txt', (err, data) => {
  console.log(data);
});
```

All these run OUTSIDE JavaScript's single thread!

7. Real-World Problem & Solution

Problem: Blocking Code

```
function fetchUserSync() {
  // Imagine this takes 3 seconds
  let result;
  // ... blocking network call
  return result;
}

console.log('Start');
const user = fetchUserSync(); // FREEZES for 3 seconds
console.log('End');
```

Solution: Non-Blocking with Web API

```
function fetchUserAsync() {
  return fetch('https://api.example.com/user');
}

console.log('Start');
fetchUserAsync().then(user ⇒ {
  console.log('Got user:', user);
});
console.log('End'); // Runs immediately!

// Output:
// Start
// End
// (later) Got user: {...}
```

8. Visualizing the Architecture

```
// JavaScript Engine (V8, SpiderMonkey)
// ├── Call Stack (single thread)
// └── Memory Heap

// Browser/Node.js Runtime
// ├── Web APIs (separate threads!)
// │   ├── setTimeout/setInterval
// │   ├── fetch/XMLHttpRequest
// │   ├── DOM events
// │   └── FileSystem
// └── Event Loop
//     ├── Microtask Queue (Promises)
//     └── Macrotask Queue (setTimeout, I/O)
```

Key Takeaways

1. **Single-threaded:** JavaScript has ONE call stack

2. **Synchronous:** Code runs line-by-line, blocking
3. **Web APIs:** Browser provides async operations (timers, network, events)
4. **Event Loop:** Checks if call stack is empty, then moves tasks from queues
5. **Queues:** Microtasks (Promises) run before Macrotasks (setTimeout)

This is why JavaScript can handle async operations despite being single-threaded - it delegates to the browser/runtime!

Javascript Doesn’t wait for Anything

JavaScript DOESN'T know and DOESN'T care!

JavaScript just calls the function and moves on. The **browser** decides what happens next.

JavaScript's Perspective:

```
// From JavaScript's point of view, ALL function calls look the same:

const result1 = Math.random();                  // Call function
const result2 = document.getElementById('btn');   // Call function
const result3 = setTimeout(() => {}, 1000);       // Call function
const result4 = fetch('/api/data');             // Call function

// JavaScript just:
// 1. Calls the function
// 2. Gets whatever is returned
// 3. Moves to next line

// JavaScript NEVER waits!
```

The Key: Return Value vs Callback

Synchronous (returns value immediately):

```
// JavaScript calls function
const width = window.innerWidth;

// Browser returns value immediately
console.log(width);  // 1920

// JavaScript uses it right away
```

Asynchronous (returns "ticket", result comes later):

```
// JavaScript calls function
const timerId = setTimeout(() => {
  console.log('Later');
}, 1000);

// Browser returns ID immediately
console.log(timerId);  // 1 (not the result!)

// JavaScript continues immediately
// Result comes later via callback
```

JavaScript NEVER Waits:

```
console.log('1');

// JavaScript doesn't know this is "async"
// It just calls it and gets a return value
const timerId = setTimeout(() => {
  console.log('3');
}, 1000);
```

```
console.log('2', timerId);  // timerId is returned immediately!

// Output:
// 1
// 2 1
// (1 second later)
// 3


// JavaScript NEVER stopped!
// It called setTimeout, got timerId (123), moved on
```

Proof: JavaScript Doesn't "Wait" for Anything

```
console.log('A');

// This looks like it should wait, but it doesn't!
fetch('/api/data');

console.log('B');  // Runs immediately!

// Output:
// A
// B
// (JavaScript never waited!)


// fetch() returns a Promise immediately
const promise = fetch('/api/data');
console.log(promise);  // Promise { <pending> }

// JavaScript got SOMETHING back (a Promise)
// And moved on immediately
```

How It Actually Works:

Example 1: Sync

```
// JavaScript's view:
const btn = document.getElementById('btn');
// ↓
// Call function → Get return value → Use it

// What happened:
// 1. JavaScript: "Call getElementById"
// 2. Browser: "Here's the element" (returns immediately)
// 3. JavaScript: "Got it, moving on"
```

Example 2: Async

```
// JavaScript's view:
setTimeout(() => console.log('Hi'), 1000);
// ↓
// Call function → Get return value (timer ID) → Use it

// What happened:
// 1. JavaScript: "Call setTimeout"
// 2. Browser: "Here's timer ID: 123" (returns immediately)
// 3. JavaScript: "Got it, moving on"
// 4. (Browser separately handles the timer)
// 5. (1 second later, browser puts callback in queue)
// 6. (Event loop brings callback back to JavaScript)
```

JavaScript's "Contract" with Functions:

```
// ALL functions work the same way from JS perspective:
```

```
// 1. Call function
// 2. Get return value (might be undefined, Promise, number, etc.)
// 3. Continue immediately

function sync() {
  return 42;
}

function async() {
  return new Promise(resolve => {
    setTimeout(() => resolve(42), 1000);
  });
}

// JavaScript calls both the SAME way:
const a = sync();    // Gets 42
const b = async();   // Gets Promise (not 42!)

// JavaScript doesn't "know" or "care" which is which
// It just gets what the function returns
```

The Real Mechanism:

```
// Synchronous:
const result = someSyncFunction();
console.log(result);  // Value is here!

// ↓ Function returns the actual value
// ↓ JavaScript has it immediately


// Asynchronous (Callback pattern):
someAsyncFunction(() => {
  console.log('Result here!');  // Value comes later
```

```
});

// ↓ Function returns immediately (maybe undefined)
// ↓ JavaScript continues
// ↓ Callback is executed LATER when result is ready


// Asynchronous (Promise pattern):
const promise = someAsyncFunction();
promise.then(result => {
  console.log(result);  // Value comes later
});

// ↓ Function returns Promise immediately
// ↓ JavaScript continues
// ↓ .then() callback is executed LATER when result is ready
```

Visual Timeline:

```
console.log('1');
const id = setTimeout(() => console.log('3'), 0);
console.log('2');

// Timeline from JavaScript's perspective:

// Time 0ms:
// Call Stack: console.log('1')
// Output: "1"

// Time 1ms:
// Call Stack: setTimeout(...)
// Return value: 1 (timer ID)
// JavaScript: "Got return value, moving on"
```

```
// Time 2ms:
// Call Stack: console.log('2')
// Output: "2"

// Time 3ms:
// Call Stack: (empty)
// JavaScript: "I'm done!"

// (Meanwhile, browser timer finishes)

// Time 10ms:
// Browser: "Hey JavaScript, here's a callback for you"
// Call Stack: console.log('3')
// Output: "3"
```

JavaScript Doesn't Distinguish:

```
// To JavaScript, these all look the same:

const a = add(2, 3);          // Call → Return → Continue
const b = Math.random();      // Call → Return → Continue
const c = window.innerWidth;  // Call → Return → Continue
const d = setTimeout(fn, 1000);// Call → Return → Continue
const e = fetch('/api');      // Call → Return → Continue

// JavaScript ALWAYS continues immediately!
// The difference is in WHAT is returned:
// a = 5 (the actual result)
// b = 0.234 (the actual result)
// c = 1920 (the actual result)
// d = 123 (timer ID, NOT the result!)
// e = Promise (NOT the result!)
```

How Async Functions Tell You "Result Later":

Method 1: Callback

```
// The function says "I'll call YOU when done"
setTimeout(() => {
  console.log('Result!');  // Called later
}, 1000);

// JavaScript never waits, callback is called later
```

Method 2: Promise

```
// The function says "Here's a Promise, use .then()"
const promise = fetch('/api/data');

promise.then(result => {
  console.log('Result!', result);  // Called later
});

// JavaScript never waits, .then() callback called later
```

Method 3: async/await (syntactic sugar)

```
// Looks like waiting, but it's just Promise syntax!
const result = await fetch('/api/data');

// Behind the scenes, this is:
// fetch('/api/data').then(result => {
//   // continue here
// });

// JavaScript still doesn't "wait"!
// The function pauses, but JavaScript engine continues with other tasks
```

The Pattern:

```
// SYNC functions:
// - Return the actual result
// - JavaScript uses it immediately

const result = syncFunction();
console.log(result);  // Have it!


// ASYNC functions:
// - Return a "placeholder" (Promise/ID/undefined)
// - Provide callback mechanism for actual result

const placeholder = asyncFunction(callback);
// OR
asyncFunction().then(callback);

// JavaScript doesn't know the difference!
// It just handles whatever is returned
```

Practical Example:

```
// JavaScript's execution (never stops):

console.log('Start');                 // Execute
const el = document.getElementById('btn'); // Execute → Get element
console.log(el);                      // Execute → Use element
const id = setTimeout(() => {}, 1000);   // Execute → Get timer ID
console.log(id);                      // Execute → Use timer ID
const p = fetch('/api');                 // Execute → Get Promise
console.log(p);                       // Execute → Use Promise
console.log('End');                   // Execute
```

```
// Output:
// Start
// <button id="btn">...</button>
// 1
// Promise { <pending> }
// End

// JavaScript NEVER paused!
// It got return values and kept going
```

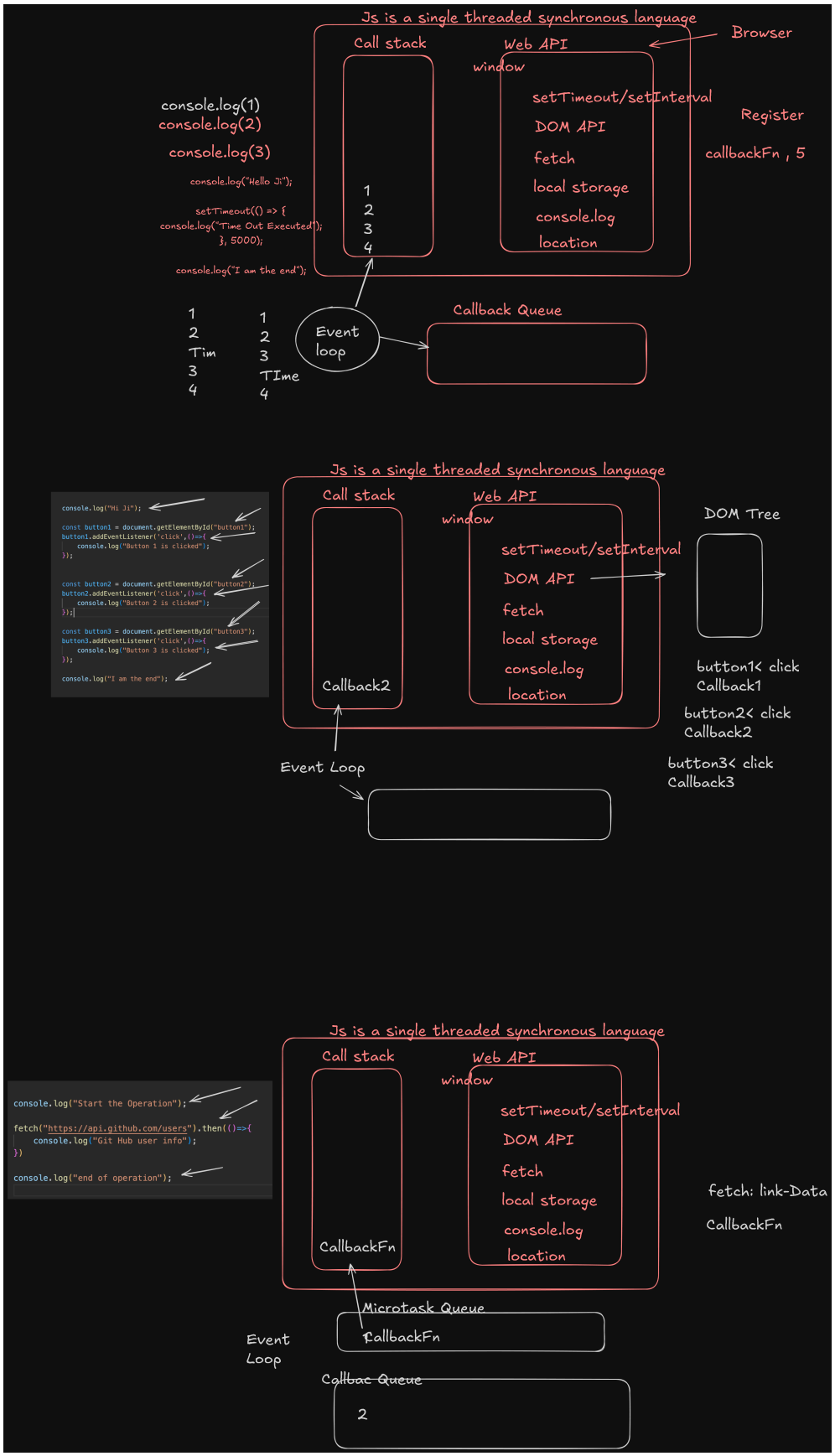
Summary:

```
// JavaScript's job:
// 1. Execute code line by line
// 2. Call functions
// 3. Get return values
// 4. Move to next line

// JavaScript NEVER decides to "wait"
// It ALWAYS continues immediately

// The function's implementation decides:
// - Sync: Return actual value
// - Async: Return placeholder, send result via callback later

// JavaScript just deals with whatever it gets!
```

JavaScript doesn't know or care if something is sync/async. It just calls functions and handles return values. The function's implementation (in browser) determines the behavior! 🎯

An API (Application Programming Interface) is ANY way for code to talk to other code.

- **Interface** - How to interact with it
- **Abstraction** - You don't need to know HOW it works internally
- **Contract** - What to send, what you'll get back


Js is a single threaded synchronous language
Call stack
Web API
window
Browser
setTimeout/setInterval
DOM API
fetch
local storage
console.log
location
Register
callbackFn , 5
console.log(1)
console.log(2)
console.log(3)
console.log("Hello Ji");
setTimeout(() => {
console.log("Time Out Executed");
}, 5000);
console.log("I am the end");
1
2
3
4
Event loop
Callback Queue
1
2
Tim
3
4
1
2
3
TIme
4
Js is a single threaded synchronous language
Call stack
Web API
window
DOM Tree
setTimeout/setInterval
DOM API
fetch
local storage
console.log
location
Callback2
button1< click
Callback1
button2< click
Callback2
button3< click
Callback3
Event Loop
console.log("Hi Ji");
const button1 = document.getElementById("button1");
button1.addEventListener('click',()=>{
console.log("Button 1 is clicked");
});
const button2 = document.getElementById("button2");
button2.addEventListener('click',()=>{
console.log("Button 2 is clicked");
});
const button3 = document.getElementById("button3");
button3.addEventListener('click',()=>{
console.log("Button 3 is clicked");
});
console.log("I am the end");
Js is a single threaded synchronous language
Call stack
Web API
window
setTimeout/setInterval
DOM API
fetch
local storage
console.log
location
CallbackFn
fetch: link-Data
CallbackFn
Microtask Queue
CallbackFn
Event Loop
Callbac Queue
2
console.log("Start the Operation");
fetch("https://api.github.com/users").then(()=>{
console.log("Git Hub user info");
})
console.log("end of operation");